针对AI辅助在线编程评估平台的架构设计与模块规范

为了满足此类系统广泛的功能需求——从基于角色的访问控制和基于浏览器的集成开发环境（IDE），到安全的代码执行和高保真遥测——架构必须进行精心划分。该领域的单体设计会引入严重的安全漏洞，特别是涉及不可信代码的执行以及保护专有系统提示免受注入攻击。因此，该系统在结构上分为四个平衡、不重叠的模块。每个模块充当独立的限界上下文或微服务，仅通过严格定义的应用程序编程接口（API）契约和JSON数据模式进行交互。这种划分确保了功能不重叠、安全边界得到维护，并且可以为代码评估等计算密集型任务独立实现水平扩展。

模块 1：用户身份与评估管理系统

用户身份与评估管理系统作为基础数据持久层和平台编排的权威来源。它完全封装了与身份验证、基于角色的访问控制（RBAC）、评估编写、会话生命周期管理以及评估结果的存档存储相关的逻辑。通过隔离这些职责，架构保证了敏感数据（例如隐藏的测试用例和候选人汇总分数）与前端客户端和代码执行环境完全隔离。

运行范围与功能边界

该模块的主要功能领域涉及安全管理多个利益相关者角色的独特交互，主要是管理员（或讲师）和候选人（或学生）。[3] 管理员需要高度结构化的界面来编写评估。这一编写过程包括用Markdown定义整体问题描述，配置允许的编程语言（特别是Python和JavaScript/TypeScript），注入初始启动代码，更重要的是，建立隐藏的计算测试用例，提交的脚本将根据这些用例进行评估。[3] 由于该系统旨在评估AI辅助工作流，编写参数还必须定义AI辅助的约束条件，确定候选人是否有权访问实时代码补全、概念性聊天或两者兼而有之。

对于候选人来说，该模块充当守门人的角色。它必须强制执行严格的会话管理，确保候选人只能在预定义的时间窗口内访问评估，并防止单个用户实体有多个并发活动会话。该模块利用强大的关系型数据库管理系统来跟踪活动会话的状态，记录启动时间戳以准确执行时间限制。这种权威的计时至关重要，因为客户端时钟很容易被操纵；因此，后端必须严格控制倒计时，并在过期时自动锁定评估。此外，该模块作为汇总评估报告的最终目的地，将执行沙盒生成的二进制执行结果（通过/失败）与AI辅助模块生成的定性语义遥测相结合，生成全面的评估配置文件。[3]

API 契约与数据模式

与用户身份和评估管理系统的通信通过RESTful HTTP API进行，利用针对正式JSON Schema定义验证的JSON结构。[4] 这确保了结构一致性，并防止格式错误的数据破坏持久层。下表描述了促进前端客户端与后端权威系统之间交互的主要API契约。

API 端点	HTTP 方法	载荷模式	响应模式	描述

/api/v1/sessions/initiate	POST	SessionInitSchema	SessionStateSchema	验证候选人凭据并初始化安全的评估会话，返回加密令牌。
/api/v1/assessments/{id}/context	GET	无	AssessmentContextSchema	检索问题描述、语言限制和启动代码，明确剥离隐藏的测试用例。
/api/v1/submissions/autosave	PUT	WorkspaceStateSchema	AcknowledgmentSchema	持久化候选人编辑器的持续状态，以防止因客户端断开连接而导致数据丢失。
/api/v1/reports/aggregate/{id}	GET	无	EvaluationReportSchema	允许管理员检索包含执行分数和AI遥测的最终合成报告。

为了明确展示数据边界的强制执行，必须严格定义 AssessmentContextSchema，以确保没有评估逻辑泄露给客户端。该模式严格规定了传输以初始化浏览器IDE的参数。

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "AssessmentContextSchema",
  "type": "object",
  "properties": {
    "assessment_id": { "type": "string", "format": "uuid" },
    "title": { "type": "string" },
```

```
    "problem_description_markdown": { "type": "string" },
    "language_constraints": {
      "type": "array",
      "items": { "type": "string", "enum": ["python", "javascript", "typescript"] }
    },
    "workspace_initialization": {
      "type": "object",
      "additionalProperties": { "type": "string", "description": "文件名到启动代码字符串的键值映射" }
    },
    "session_expiration_timestamp": { "type": "string", "format": "date-time" }
  },
  "required": ["assessment_id", "title", "problem_description_markdown", "language_constraints",
"workspace_initialization"]
}
```

严格的关注点分离要求当前端查询 /api/v1/assessments/{id}/context 时，后端动态映射数据库记录，故意丢弃验证测试用例数组。这保证了针对客户端网络流量的恶意逆向工程不会给候选人带来任何优势。

模块 2：交互式基于浏览器的协作空间（前端IDE）

交互式基于浏览器的协作空间构成了呈现给候选人的整个视觉和交互层。该模块围绕Monaco Editor设计，提供了一个丰富、响应迅速且熟悉的创作环境，无需本地环境设置要求。[3] 它负责捕获击键、管理虚拟文件、与AI遥测服务交互以实现代码补全，并安全地将代码传输到后端执行。

运行范围与功能边界

该模块的架构核心是Monaco Editor，这是一款极其强大的文本编辑器，也是Visual Studio Code的基础。[5] 此实现超越了仅仅挂载一个通用的文本区域；它需要对Monaco的底层概念进行复杂的操作系统，特别是模型（Models）和统一资源标识符（URIs）。单个候选人会话通常涉及操作多个文件。该模块必须利用 monaco.editor.createModel 实例化虚拟文件系统，为每个打开的文件分配唯一的URI。[5] 这种结构使编辑器能够管理不同的文本内容，应用适当的特定语言语法高亮显示，并跟踪独立的编辑历史记录。[5]

一个关键的操作要求是无缝保存正在进行的代码，以降低因局部网络不稳定导致灾难性数据丢失的风险。[3] 虽然多人实时协作引擎通常依赖于通过WebSockets交付的强大操作转换（OT）或无冲突复制数据类型（CRDT）——如在Figma或Discord等大型平台中所见——但该评估平台明确地将实时协作排除在其范围之外。[3] 因此，仅依靠WebSockets会引入不必要的架构脆弱性；断开的连接可能会在没有任何提示的情况下无法同步状态。[7] 相反，该模块实施了高度可靠的客户端防抖自动保存（debounced autosave）策略。[8] 每次Monaco ITextModel 的变动都会触发一个被放入防抖队列的事件。在用户不活动时，状态会被序列化，并通过标准的HTTP PUT请求传输到用户身份模块。[9] 关键是，当需要对编辑器进行编程式更新时（例如注入启动代码或应用AI建议的重构），该模块必须使用 pushEditOperations 而不是 setValue，从而确保候选人的撤销/重做堆栈保持完美

无缺。[10]

AI辅助的集成代表了前端模块中技术最复杂的方面。IDE必须通过提供内联的“幽灵文本（ghost text）”补全来模拟高级企业工具的行为。[11] 这是通过利用Monaco的registerInlineCompletionsProvider API来实现的。[12] 当候选人键入时，提供程序会捕获当前的代码上下文。为防止耗尽平台的API速率限制并产生不可控的Token成本，该模块必须在provideInlineCompletions 方法上实现积极的速率限制和防抖算法，确保仅当候选人明确暂停打字时，才会向AI服务发送网络请求。[13] 当AI服务响应时，IDE将建议渲染为幽灵文本，允许用户通过标准键绑定接受补全，从而无缝地将建议合并到 ITextModel 中。[14]

API 契约与接口规范

前端IDE作为一个无状态客户端运行。它从不直接与数据库或执行沙盒通信，强制执行严格的零信任边界。它仅通过定义的API契约与后端模块进行交互。

契约接口	目标模块	**HTTP 方法**	载荷模式	描述
SyncWorkspaceState	身份模块	PUT	WorkspaceStateSchema	传输所有活动Monaco模型的序列化字符串内容以进行持久的自动保存。
RequestInlineCompletion	AI服务	POST	IDEContextSchema	将周围的代码上下文和光标坐标传输到AI代理以生成生成式文本补全。
ExecuteEvaluation	身份模块	POST	ExecutionTriggerSchema	指示后端冻结当前工作区状态并将其分派到沙盒以针对隐藏的测试用例进行正式评估。

IDEContextSchema 对于为AI遥测服务提供足够的局部感知以生成精确的语法预测（而无需传输整个项目仓库）至关重要，从而优化了负载大小和处理延迟。

JSON

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "IDEContextSchema",
 "type": "object",
 "properties": {
  "session_id": { "type": "string", "format": "uuid" },
  "active_uri": { "type": "string", "format": "uri" },
  "cursor_metrics": {
   "type": "object",
   "properties": {
    "line_number": { "type": "integer" },
    "column": { "type": "integer" }
   }
  },
  "prefix_context": { "type": "string", "description": "光标前代码的字符串表示" },
  "suffix_context": { "type": "string", "description": "光标后代码的字符串表示" }
 },
 "required": ["session_id", "active_uri", "cursor_metrics", "prefix_context"]
}
```

通过严格定义这些参数，架构确保前端保持非常轻量级，将语言模型提示工程和安全清理等计算繁重的任务卸载给下游的AI服务模块。

模块 3：沙盒代码执行与评估引擎

安全编译、执行和评估外部候选人提交的任意不可信代码的能力，构成了平台架构中最关键的安全漏洞向量。沙盒代码执行与评估引擎作为一个高度隔离、戒备森严的子系统运行，在结构上与主要的Web应用程序路由和用户数据库断开连接。[3] 该模块的唯一职责是接收代码载荷，在极其严格的资源限制下执行它，并返回确定性的评估结果。

运行范围与功能边界

评估源代码需要深度隔离，以减轻系统性风险，例如远程代码执行（RCE）利用、本地网络侦察、权限提升以及通过无限循环或内存炸弹发起的拒绝服务（DoS）攻击。[15] 虽然将此责任委托给无服务器云函数（如AWS Lambda或Google Cloud Run）提供了理论上的隔离，但“冷启动”带来的延迟以及与高频评估相关的极其昂贵的财务成本（例如，在大学并发考试期间每秒处理数千个请求）决定了必须采用定制的、基于Kubernetes的自托管架构。[16]

这种架构反映了企业级开源执行平台（如Judge0）建立的稳健范式。[17] 为了在实现几乎瞬时的执

行时间的同时获得坚不可摧的安全性，该模块避开了笨重的虚拟机（VM），转而采用深度容器化技术，特别是利用 nsjail。[19] nsjail 机制通过组合多种Linux内核安全功能来创建隔离环境。[19] 它大量使用命名空间——特别是进程ID（PID）、挂载（mount）、网络（network）和用户（user）命名空间——来诱导执行的脚本相信它是独立操作系统上运行的唯一进程。关键是，该模块实现了seccomp-bpf（使用Berkeley包过滤器的安全计算）来过滤并限制沙盒进程可以执行的系统调用。[19] 例如，对 execve 的调用会被拦截并阻止，从而防止恶意代码生成辅助shell或访问bash环境。[20] 网络访问被完全切断，使得数据泄露成为不可能。[15]

同时，平台利用内核控制组（cgroups）对系统资源强制执行严格的配额。[19] 如果候选人提交的Python脚本进入无限的 while True: 循环，cgroup监视器将在它超过预定义的 wall_time_limit 或 cpu_time_limit 的那一毫秒暴力终止执行进程，并返回“超出时间限制”的状态代码。[21]

运行执行生命周期是严格按顺序且异步的：

1. 队列摄取：用户身份系统将包含候选人代码和隐藏测试用例的组合载荷分派到异步消息代理（例如 Redis 或 RabbitMQ）中。[15] 这确保了即使在流量激增期间，主要API仍能保持高度响应。
2. 资源调配：可用的工作节点使用该任务，调配短暂的文件系统。如果提交涉及多个文件（例如，导入实用程序的JavaScript模块），该模块会处理提取包含 additional_files 的压缩目录结构的过程。[22]
3. 编译与执行：相应的语言运行时（例如 CPython 或 Node.js）在受限的 nsjail 包装器中被调用，应用特定的内存和时间限制。[15] 替代架构可以利用 WebAssembly (Wasm) 运行时（如 Deno 或 Pyodide）实现极致轻量级的执行，尽管容器化执行仍然是支持完整语言特性的企业标准。[23]
4. 指标聚合：在进程终止时，工作节点捕获标准输出 (stdout)、标准错误 (stderr)、退出状态代码以及精确的计算指标，包括总CPU时间和峰值内存消耗。[22]
5. 输出评估：工作节点根据测试用例载荷中提供的 expected_output，对捕获的 stdout 进行确定性字符串或正则表达式匹配，计算最终的布尔结果。[16]
6. 破坏性清理：短暂的文件系统被永久删除，容器实例被销毁，保证持久状态或残留的恶意载荷不会污染后续的执行周期。[15]

API 契约与执行模式

为了确保绝对的网络隔离，执行引擎永远不会暴露在公共互联网上；它只处理由身份模块发送的内部网络流量。API模式大量复制了Judge0 API利用的高度优化的JSON结构。[21]

参数属性	数据类型	必须	架构目的
source_code	Base64 字符串	是	原始候选人脚本，已编码以防止 JSON 解析错误。

language_id	整数	是	映射到特定编译器/运行时的数字标识符。
additional_files	Base64 ZIP	否	用于复杂的多文件架构问题评估。[22]
expected_output	字符串	否	引擎用于验证以确定通过/失败条件的明确字符串。[24]
cpu_time_limit	浮点数	否	在内核终止之前分配的绝对最大处理时间。[21]

输出模式标准化了执行成功或失败的无数种方式，确保身份模块能够确定性地解析结果，而无需深入了解特定于运行时的堆栈跟踪。

JSON

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "ExecutionResultSchema",
 "type": "object",
 "properties": {
  "execution_token": { "type": "string", "description": "执行运行的唯一加密标识符" },
  "status_evaluation": {
   "type": "object",
   "properties": {
    "status_id": { "type": "integer" },
    "status_description": { "type": "string", "enum": }
   }
  },
  "captured_stdout": { "type": ["string", "null"] },
  "captured_stderr": { "type": ["string", "null"] },
  "computational_metrics": {
   "type": "object",
   "properties": {
    "cpu_time_seconds": { "type": "number" },
    "peak_memory_kb": { "type": "integer" }
```

```
    }
  }
 },
 "required": ["execution_token", "status_evaluation", "computational_metrics"]
}
```

这种严格的 JSON 契约使“用户身份模块”能够快速提取执行结果，更新候选人的评估状态，并随后将通知推送回前端客户端，从而将繁重的处理层与用户体验流完全解耦。

模块 4：人工智能遥测与辅助服务

为了实现平台全面评估AI辅助工作流的首要目标，架构需要一个复杂的专属服务层来调解、规范和分析候选人与外部大型语言模型 (LLMs) 之间的所有交互。[3] 该模块具有双重功能：既是安全的代理执行网关，也是用于高级语义遥测的高吞吐量数据摄取管道。[25]

运行范围与功能边界

允许基于浏览器的客户端直接与专有 LLM API（例如 OpenAI 或 Anthropic）通信会带来灾难性的安全和财务风险。怀有恶意的候选人可以轻易拦截网络流量以提取企业 API 密钥，绕过速率限制将平台用于个人生成任务，或者利用提示注入向量泄露平台的专有系统提示和评估标准——这种漏洞被正式归类为 OWASP LLM07:2025 系统提示泄露 (System Prompt Leakage)。[26] 因此，AI 遥测服务作为一个不透明的后端代理运行。前端 IDE 仅传输原始代码上下文和用户意图。AI 服务将此上下文包装在经过强化的服务器端系统提示中，明确限制模型的行为，指示它充当教育导师而不是自治的代码生成器，然后再将载荷分派给外部 LLM 提供商。

此外，该模块还负责缓解AI辅助工作流程中固有的特定漏洞，最著名的是“包幻觉攻击 (Package Hallucination Attacks)”。[27] 因为生成式模型偶尔会合成完全虚构的软件包，所以AI服务管道必须在将建议中继回候选人的IDE之前，对照已知良好的包注册表验证建议的导入，从而防止候选人合并不存在的或恶意的幻觉依赖项。[27]

一个关键的架构复杂性涉及确保LLM返回Monaco编辑器可以无缝渲染的编程式格式的数据。如果IDE期望收到用于内联“幽灵文本”的原始字符串时，LLM返回的是对话式自然语言，那么前端逻辑将会失败。AI服务通过大量利用约束解码 (Constrained Decoding) 方法和JSON模式，明确强制执行确定性输出。[28] 通过将严格的JSON模式直接注入到LLM请求载荷中，AI服务在算法上迫使模型根据预定义参数构建其响应。[30]

同时，该模块也是遥测聚合核心。为了准确评估候选人如何使用AI，标准指标是不够的；系统需要精细的、标准化的跟踪。该架构实现了针对LLM工作负载进行了深度定制的 OpenTelemetry (OTLP) 规范。[32] 每次交互都会生成一个OTLP跨度 (span)，捕获详尽的元数据：特定的模型标识符、交互的精确时间戳、提示的文本内容以及Token消耗的严格细分（区分提示上下文Token和生成的补全Token）。[32] 这些跨度由 BatchSpanProcessor 缓冲并导出到中央数据库，确保在高流量会话期间不会丢失任何数据。[35]

除了原始元数据收集之外，该服务还执行高级语义遥测 (Semantic Telemetry)。[25] 该模块将候选人的聊天日志异步传递给一个较小的二级分类模型，而不是强迫讲师手动阅读数百份聊天记录来评

估候选人的解决问题能力。该模型为交互分配语义分类标签。[25] 如果候选人反复要求AI从头开始编写整个函数，则遥测被标记为“过度依赖/代码生成”。相反，如果候选人提交了回溯日志 (traceback) 并要求解释特定变量的状态，则该交互被标记为“诊断调试”。这些语义标签是最终评估报告的基础，彻底将评估从“代码起作用了吗？”转变为“工程师是否有效地与他们的工具协作了？”

API 契约与遥测模式

遥测模式大量利用标准化的 OTLP 字典来确保交叉兼容性和详细的审计。从 AI 服务传输到内部数据库进行存档的载荷代表了一种高度丰富的数据结构。

遥测属性	数据类型	上下文用途
llm.token_count.total	整数	用于财务审计和评估候选人的效率。[34]
llm.request.model_name	字符串	标识用于跟踪的特定模型（例如，gpt-3.5-turbo-0125）。[34]
interaction.semantic_tag	枚举字符串	由语义遥测引擎生成的意图分类。[25]
interaction.prompt_text	字符串	候选人提交的确切自然语言字符串。[32]

为了对外部 LLM 提供商执行约束解码，确保响应可以由前端 IDE 以编程方式呈现，AI 服务动态生成以下 JSON 模式并将其与系统提示一起传输。[31]

JSON

```
{
 "name": "structured_inline_completion",
 "strict": true,
 "schema": {
  "type": "object",
  "properties": {
   "internal_reasoning": {
    "type": "string",
    "description": "AI 逐步阐述此代码为何能解决提示要求的逻辑。"
```

```
    },
    "monaco_insert_text": {
      "type": "string",
      "description": "将直接注入 IDE 的确切语法代码字符串。"
    },
    "hallucination_check_passed": {
      "type": "boolean",
      "description": "内部模型验证标志，确保没有导入虚构的包。"
    }
  },
  "required": ["internal_reasoning", "monaco_insert_text", "hallucination_check_passed"],
  "additionalProperties": false
 }
}
```

收到这种结构化响应后，AI 代理提取 monaco_insert_text 属性，并将其传送回候选人的浏览器，以便立即渲染为内联幽灵文本。internal_reasoning 属性被故意从客户端载荷中剥离，仅路由至遥测数据库，为管理员提供 AI 用于协助学生的底层逻辑的深入洞察，从而完成全面的评估闭环。

模块间数据流与边界强制保证

该平台的架构完整性从根本上依赖于在四个模块之间严格执行通信边界。为了明确确保没有功能重叠且没有未被缓解的安全漏洞，在关键操作上下文中数据以单向流动，并受制于精确的访问控制。

1. 前端 **IDE** 到数据库的隔离：Monaco 编辑器环境（模块 2）对底层的持久层（模块 1）一无所知。所有状态同步都通过经过身份验证的 REST API 端点进行。该架构正式保证在任何情况下都不会将隐藏的评估测试用例传输到客户端环境。[3] 因此，企图拦截 WebSocket 流量或检查客户端网络载荷以提取答案的敌对尝试在数学上注定会失败。
2. 执行触发与编排边界：浏览器客户端（模块 2）不具备通往沙盒执行引擎（模块 3）的直接网络路由。当候选人发起“运行代码”命令时，前端仅将原始源代码传输到身份模块（模块 1）。后端在服务器端安全地将用户代码与隐藏的验证脚本连接起来，然后才将复合载荷分派给隔离的沙盒。[16] 这种编排确保执行环境仅处理由权威的身份模块验证过的请求，从而防止未经授权的操作者滥用计算资源昂贵的沙盒。
3. 遥测聚合隔离：前端 IDE 仅通过 AI 遥测服务（模块 4）请求 AI 协助。通过将 AI 代理逻辑与主要用户数据库隔离开来，平台保证了生成语义遥测分类和缓冲 OpenTelemetry 跨度等计算密集型任务不会对用户身份系统引入延迟。AI 服务处理与外部 LLM 交互的繁重工作，并随后将异步日志事件发送回身份数据库，从而在整个系统层次结构中保持高响应性。[25]

架构韧性与可扩展性约束

该设计经过明确构建，能够容忍各个微服务内部的灾难性故障，而不会引发系统性的平台停机。[3]

由于沙盒执行引擎（模块 3）建立在通过 nsjail 和消息代理实现的短暂的、深度隔离的容器化基础

之上，因此它可以优雅地管理并发执行请求的剧增——这是在测试窗口结束时，数百名候选人同时提交评估的典型场景。[15] 工作节点可以水平扩展以处理队列，而不会使前端或数据库过载。如果队列饱和，执行可能会延迟，但用户身份系统（模块 1）保持高度响应，允许候选人继续与其编辑器交互并保存他们的状态。

类似地，AI 遥测服务（模块 4）包含了稳健的优雅降级 (graceful degradation) 协议。如果外部 LLM 提供商发生全面中断，代理服务会立即返回一个标准化的后备模式，表明服务暂时不可用。前端 IDE 解析此特定模式，禁用 Monaco 编辑器中的内联补全和聊天界面，并向候选人发出警报。由于这些模块是独立且不重叠的，因此 AI 服务的故障对候选人编写代码、在沙盒中执行测试或正式提交其评估的能力绝对没有影响。[3]

对模块化解耦的绝对坚持，使用 JSON 模式对 API 契约的严格定义，以及执行沙盒内毫不妥协的安全性，直接满足了评估现代 AI 辅助软件工程能力的复杂功能和非功能需求。该平台成功地将评估重点从孤立的算法记忆转移到了对人机协作工作流程的全面、数据驱动的评估上。

Works cited

1. CoderPad: Coding Interview & Technical Assessment Platform, accessed April 30, 2026, https://coderpad.io/
2. The Best AI Coding Assistants: A Full Comparison of 17 Tools, accessed April 30, 2026, https://axify.io/blog/the-best-ai-coding-assistants-a-full-comparison-of-17-tools
3. requirement.md
4. A Media Type for Describing JSON Documents - JSON Schema, accessed April 30, 2026, https://json-schema.org/draft/2020-12/json-schema-core
5. microsoft/monaco-editor: A browser based code editor - GitHub, accessed April 30, 2026, https://github.com/microsoft/monaco-editor
6. How do autosave features (like Medium/Notion) actually work at scale? : r/webdev - Reddit, accessed April 30, 2026, https://www.reddit.com/r/webdev/comments/1nfatcs/how_do_autosave_features_like_mediumnotion/
7. Best Practices: Reliable Auto-Save in TX Text Control Using the WebSocketHandler and Background Services, accessed April 30, 2026, https://www.textcontrol.com/blog/2025/06/20/best-practices-reliable-auto-save-in-tx-text-control-using-the-websockethandler-and-background-services/
8. To save or to autosave: Autosaving patterns in modern web applications | by Brooklyn Dippo, accessed April 30, 2026, https://medium.com/@brooklyndippo/to-save-or-to-autosave-autosaving-patterns-in-modern-web-applications-39c26061aa6b
9. What is the preferred way of persisting data across web socket connections in server memory? - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/70826489/what-is-the-preferred-way-of-persisting-data-across-web-socket-connections-in-se
10. Building a code editor with Monaco | by Satyajit Sahoo - Exposition, accessed April 30, 2026,

https://blog.expo.dev/building-a-code-editor-with-monaco-f84b3a06deaf
11. Best AI for coding? : r/ChatGPTCoding - Reddit, accessed April 30, 2026, https://www.reddit.com/r/ChatGPTCoding/comments/1icgojf/best_ai_for_coding/
12. Custom Inline completions with built-in Suggest widget · microsoft monaco-editor · Discussion #3917 - GitHub, accessed April 30, 2026, https://github.com/microsoft/monaco-editor/discussions/3917
13. monaco-editor: limit api calls on registerInlineCompletionsProvider - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/76831759/monaco-editor-limit-api-calls-on-registerinlinecompletionsprovider
14. Monaco: How to add inline autocomplete/code suggestion? - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/70079620/monaco-how-to-add-inline-autocomplete-code-suggestion
15. Designing LeetCode: Online Code Judge System - DEV Community, accessed April 30, 2026, https://dev.to/matt_frank_usa/designing-leetcode-online-code-judge-system-4372
16. Design an online judge like Leetcode | by Dilip Kumar - Medium, accessed April 30, 2026, https://dilipkumar.medium.com/design-an-online-judge-like-leetcode-30ff9e73b248
17. GitHub - judge0/judge0: Robust, fast, scalable, and sandboxed open-source online code execution system for humans and AI., accessed April 30, 2026, https://github.com/judge0/judge0
18. Judge0 - Code Execution Made Simple for Every Business, accessed April 30, 2026, https://judge0.com/
19. restyler/awesome-sandbox: Awesome Code Sandboxing for AI - GitHub, accessed April 30, 2026, https://github.com/restyler/awesome-sandbox
20. How to implement a sandbox for an online-judge system? - Stack Overflow, accessed April 30, 2026, https://stackoverflow.com/questions/32389358/how-to-implement-a-sandbox-for-an-online-judge-system
21. Source code for judge0api.submission, accessed April 30, 2026, https://judge0api.readthedocs.io/en/latest/_modules/judge0api/submission.html
22. judge0/docs/api/submissions/submissions.md at master - GitHub, accessed April 30, 2026, https://github.com/judge0/judge0/blob/master/docs/api/submissions/submissions.md
23. Secure code execution - Hugging Face, accessed April 30, 2026, https://huggingface.co/docs/smolagents/tutorials/secure_code_execution
24. Judge0 | Documentation | Postman API Network, accessed April 30, 2026, https://www.postman.com/judge0/public/documentation/k4ahkcl/judge0
25. Semantic Telemetry: Understanding how users interact with AI systems - Microsoft Research, accessed April 30, 2026,

https://www.microsoft.com/en-us/research/blog/semantic-telemetry-understanding-how-users-interact-with-ai-systems/
26. What data do coding agents send, and where to? | Chaser Systems, accessed April 30, 2026, https://chasersystems.com/blog/what-data-do-coding-agents-send-and-where-to/
27. AI-Assisted Coding: A Practical Guide for Software Engineers - Frontend Masters, accessed April 30, 2026, https://frontendmasters.com/blog/ai-assisted-coding-a-practical-guide-for-software-engineers/
28. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models, accessed April 30, 2026, https://arxiv.org/html/2501.10868v3
29. JSON Mode - Workers AI - Cloudflare Docs, accessed April 30, 2026, https://developers.cloudflare.com/workers-ai/features/json-mode/
30. Structured model outputs | OpenAI API, accessed April 30, 2026, https://developers.openai.com/api/docs/guides/structured-outputs
31. Making AI Useful: How to Use json_schema and Function (via. tools) in the Responses API, accessed April 30, 2026, https://medium.com/@arda.arslan/making-ai-useful-how-to-use-json-schema-and-function-via-tools-in-the-responses-api-a36568ab6694
32. AI Coding Data Reference | Revenium Developer Docs, accessed April 30, 2026, https://docs.revenium.io/ai-coding-dashboard/ai-coding-data-reference
33. Use Local-First OpenTelemetry Capture for AI Coding Agent Debugging Workflows, accessed April 30, 2026, https://oneuptime.com/blog/post/2026-02-06-local-otel-ai-coding-agent-debugging/view
34. Instrumenting OpenAI with OpenTelemetry (OTel) | Engineering | ClickHouse Resource Hub, accessed April 30, 2026, https://clickhouse.com/resources/engineering/openai-otel
35. Tracing the Conversation: Open Telemetry and AI Toolkit for GenAI Performance Insights, accessed April 30, 2026, https://techcommunity.microsoft.com/blog/educatordeveloperblog/tracing-the-conversation-open-telemetry-and-ai-toolkit-for-genai-performance-ins/4471782